

REFERENCES, MEMORY USAGE, AND SYS.GETSIZEOF()

It might seem like creating a new list instead of modifying the original one wastes memory. But remember that, just as variables technically contain references to values instead of the actual values, lists also contain references to values. The `newGreetings.append(word)` line shown earlier isn't making a copy of the string in the `word` variable, just a copy of the reference to the string, which is much smaller.

You can see this by using the `sys.getsizeof()` function, which returns the number of bytes that the object passed to it takes up in memory. In this interactive shell example, we can see that the short string `'cat'` takes up 52 bytes, whereas a longer string takes up 85 bytes:

```
>>> import sys
>>> sys.getsizeof('cat')
52
>>> sys.getsizeof('a much longer string than just "cat"')
85
```

(In the Python version I use, the overhead for the string object takes up 49 bytes, whereas each actual character in the string takes up 1 byte.) But a list containing either of these strings takes up 72 bytes, no matter how long the string is:

```
>>> sys.getsizeof(['cat'])
72
>>> sys.getsizeof(['a much longer string than just "cat"'])
72
```

The reason is that a list technically doesn't contain the strings, but rather just a reference to the strings, and a reference is the same size no matter the size of the referred data. Code like `newGreetings.append(word)` isn't copying the string in `word`, but the reference to the string. If you want to find out how much memory an object, and all the objects it refers to, take up, Python core developer Raymond Hettinger has written a function for this, which you can access at <https://code.activestate.com/recipes/577504-compute-memory-footprint-of-an-object-and-its-cont/>.

So you shouldn't feel like you're wasting memory by creating a new list rather than modifying the original list while iterating over it. Even if your list-modifying code seemingly works, it can be the source of subtle bugs that take a long time to discover and fix. Wasting a programmer's time is far more expensive than wasting a computer's memory.

Although you shouldn't add or remove items from a list (or any iterable object) while iterating over it, it's fine to modify the list's contents. For example, say we have a list of numbers as strings: `['1', '2', '3', '4', '5']`. We can convert this list of strings into a list of integers `[1, 2, 3, 4, 5]` while iterating over the list: